

# State Space Analysis of a Stretch3 Robot for a Pick and Place Application

Abhinav Viswanathan, Arvind Kaushik, Reuben Kukar, Jatin Satyam

## 1 Introduction

Pick-and-place manipulation is a core application in automation, used in assembly lines, factory sortation systems, smartphone manufacturing, and other high-throughput industries such as automotive and consumer electronics. Most existing solutions rely on fixed-base industrial arms (e.g., ABB, FANUC, KUKA). These systems provide precise motion but are limited to “fixed automation” layouts: they operate within a restricted workspace and typically require high, inflexible capital investment. This project explores an alternative based on mobile manipulation. We implement a pick-and-place simulation using the Stretch3 robotic mobile manipulator to demonstrate “flexible automation,” where the robot is not confined to a fixed workcell. The Stretch3 robot performs the task in the MuJoCo physics simulator, while MATLAB is used to obtain and validate the state-space representation of the system.

## 2 Modeling

An analytically tractable mathematical model was constructed to represent two degrees of freedom relevant to the pick-and-place application: the lift of the arm and the extension/retraction of the arm. The joints of the robot are labelled  $q_1$  and  $q_2$ , driven by actuators  $a_1$  and  $a_2$  respectively.

The joints were modelled as mass–spring–damper systems, and hence the equation of motion was derived as  $m_i\ddot{q}_i + b_i\dot{q}_i + k_iq_i = \tau_i$ .

The actuator definition is given by  $\tau_i = a_i$ .

The equation of motion of each actuator is  $\dot{a}_i = -\frac{1}{T}a_i + \frac{1}{T}u_i$ .

The state vector  $x$  is defined as  $x = \begin{bmatrix} q_1 & q_2 & \dot{q}_1 & \dot{q}_2 & a_1 & a_2 \end{bmatrix}^T$ , and the input vector  $u$  is  $u = \begin{bmatrix} u_1 & u_2 \end{bmatrix}^T$ .

**Where:**

- $q_i, \dot{q}_i, \ddot{q}_i$  are the joint position, velocity, and acceleration.
- $m_i$  is the mass/inertia of joint  $i$ .
- $b_i$  is the damping coefficient of joint  $i$ .
- $k_i$  is the stiffness of joint  $i$ .
- $\tau_i$  is the torque applied to joint  $i$ .
- $\tau_i = a_i$  defines the actuator state.
- $\dot{a}_i$  describes the actuator state dynamics.
- $u_i$  is the control input for actuator  $i$ .
- $T$  is the actuator time constant.

For the purpose of constructing the state-space matrices  $A$ ,  $B$ ,  $C$ , and  $D$ , numerical values are assumed for both joints.

| <b>Joint 1 parameters:</b> | <b>Joint 2 parameters:</b> | <b>Actuator time constant:</b> |
|----------------------------|----------------------------|--------------------------------|
| • $m_1 = 1$                | • $m_2 = 1$                | • $T = 0.1$                    |
| • $b_1 = 2$                | • $b_2 = 1$                |                                |
| • $k_1 = 10$               | • $k_2 = 3$                |                                |

The resulting equations of motion are:

$$\begin{aligned} \ddot{q}_1 &= -10q_1 - 2\dot{q}_1 + a_1, & \ddot{q}_2 &= -3q_2 - \dot{q}_2 + a_2, \\ \dot{a}_1 &= -10a_1 + 10u_1, & \dot{a}_2 &= -10a_2 + 10u_2. \end{aligned}$$

## State-Space Matrices

The state-space representation is:

$$\dot{x} = f(x, u),$$

where the matrices are defined as

$$A = \frac{\partial f}{\partial x}, \quad B = \frac{\partial f}{\partial u}, \quad C = I_6, \quad D = 0.$$

**Where:**

- $n$  is the number of states.
- $m$  is the number of inputs.
- $p$  is the number of outputs.

A PD controller was designed to model and simulate all 8 Degrees of Freedom of the Stretch3 robot in MuJoCo. It drives the joints of Stretch3 using position error and joint velocity. Kp (proportional gain) acts like stiffness: increasing the Kp makes the joint respond faster, but too high can cause overshoot and oscillations; decreasing Kp makes the joint move slowly to the target position. Kd (derivative gain) acts like damping: if it's too low, the motion is bouncy with overshoot and oscillations, and if it's too high, the motion becomes overly slow and heavy, as if the joint is moving through thick friction. Kv (derivative) is the velocity gain, and is a parameter that controls both the opening/closing of the gripper as well as the wheels, and was also tuned in the PD Controller script.

The PD Control Law:

$$\tau = K_p (q_{\text{ref}} - q) - K_d \dot{q}$$

And the control law for these two joints become:

$$\tau = K_p (q_{\text{ref}} - q) - K_v \dot{q}$$

In the controller script,  $k_d = k_v$  for all joints other than the gripper and the wheels.

The PD tuning was done in such a way that the robot moves to the target with maximal speed and minimal overshoot, no sustained oscillations and good pick-and-place precision, and was implemented directly within the robot's XML configuration.

While the advantages of the PD Controller may be precise control of the robot while ensuring no oscillation or overshoot, the controller is slow. To overcome this limitation of the PD controller and therefore improve the simulation, an LQR (Linear Quadratic Regulator) was designed.

The LQR controller only modeled the two degrees of freedom - lift and extension/retraction of the arm, as a proof of concept. In this controller design, the matrices Q and R, which are the weighting matrices, are tuned to meet the expected speed requirement. The Q matrix is called the State Weighing Matrix, and the R matrix is also called the Control Weighing Matrix. The bigger weight in Q implies a tighter control on that state, which results in a faster response. The bigger weight on R results in a slower and smoother system with lesser effort.

### 3 Analysis and Properties

The state space design was implemented in MATLAB. The state variables and input variables using 'syms', along with the joint parameters and actuator time constant, with the above-mentioned numerical values. Accordingly, the equations of motions, the state vector and the input vectors were defined.

The A, B, C, and D matrices were constructed in MATLAB, providing the basis for analysing further system properties - Controllability, Observability, Stability. The controllability matrix was computed using the 'ctrb' function in MATLAB with matrices A and B as parameters. Similarly, the observability matrix was computed using the 'obsv' function with matrices A and C as the parameters. The rank of both the controllability

and observability matrices were analysed and found to be full rank, proving that the overall system was controllable and observable. Upon analysing the eigenvalues of  $A$ , it was found that all the eigenvalues had negative real parts, and hence it was concluded that the system was internally asymptotically stable, and since the system is LTI and proper, it was also concluded that the system is also BIBO stable.

## 4 Simulation Results

MuJoCo (Multi-Joint Dynamics with Contact) is a high-performance physics simulator designed specifically for robotics, biomechanics, reinforcement learning and optimal control. For the simulation of the Stretch3 robot, the MuJoCo physics simulator was used due to the above advantage.

The Stretch3 simulation for the pick and place of a couple of objects (a cylinder and a cuboid) was completed in MuJoCo, while simultaneously tuning the PD parameters in the Controller script, in an iterative manner. The simulation was done in such a way that the main degrees of freedom of the Stretch3 robot (a total of 8 DOFs) were harnessed - Base movement in the x-direction - 1 DOF, Base movement in the y-direction - 1 DOF, Prismatic joint movement of the manipulator in the y-direction - 1 DOF, Prismatic joint movement of the manipulator in the z-direction - 1 DOF, Roll/pitch/Yaw of the wrist - 3 DOFs, open/close movement of the gripper - 1 DOF.

The comparison of the PD controller and the LQR controller is seen in the following illustration for both the lift and arm extension/retraction degrees of freedom.

It was inferred that while both LQR and PD controllers, there was a small undershoot in the case of the LQR controller modeling the lift DOF. The settling time of the LQR controller was found to be faster, and the control aggressiveness was found to be greater than the PD controller.

## 5 Conclusions and Discussions

This proof of concept successfully demonstrated the simulation of the Stretch3 robot performing the pick and place operation of a cuboidal object and a cylindrical object.

Subsequently, the state space representation of the Stretch3 robot was retrieved, and the analysis of the system was done in MATLAB, based on critical control system factors - Controllability, Observability, Internal Stability and BIBO stability, concluding that this system is feasible to be implemented and tested in the real world.

While the system performed satisfactorily, there are parts of the project to be improved to further develop this project. The major areas of improvement are (i) Analysis of other controller algorithms other than the already implemented PD controller and the LQR controller, such as the PID controller and the MPC controller, can be tried and tested to achieve better results in simulation, (ii) The proof of concept needs to be implemented on the Stretch3 hardware to fully validate the working of the concept.

## 6 Contributions of Team Members

Our team adopted a structured division of responsibilities to ensure smooth progress throughout the project. Each member contributed uniquely to the workflow—ranging from conceptual design and technical research to implementation and testing.

The contribution of **Abhinav Viswanathan** was the end-to-end development of the simulation in MuJoCo Simulator using Python Programming. The contribution of **Arvind Kaushik** was writing the MATLAB scripts for getting the state space equations and analysis of controllability, observability, internal stability and BIBO stability of the system. The contribution of **Reuben Kukar** was to tune the parameters of the PD controller through multiple iterations, which thereby resulted in a refined simulation. The contribution of **Jatin Satyam** was the development of the Linear Quadratic Regulator.

## 7 Appendix

```
def build_lqr_gain(A, B):
    Q = np.diag([
        500.0, # q1 position weight
        500.0, # q2 position weight
        200.0, # dq1 velocity weight
        80.0,  # dq2 velocity weight
        20.0,  # a1 actuator state
        5.0    # a2 actuator state
    ])

    R = np.diag([
        0.15, # DOF1 control effort
        0.05  # DOF2 control effort
    ])

    K_lqr, S, eigvals = control.lqr(A, B, Q, R)
    return np.array(K_lqr), eigvals
```

Figure 1: LQR controller implementation in Python

```
A =
    0    0    1    0    0    0
    0    0    0    1    0    0
   -10    0   -2    0    1    0
    0   -3    0   -1    0    1
    0    0    0    0   -10    0
    0    0    0    0    0   -10

B =
    0    0
    0    0
    0    0
    0    0
   10    0
    0   10

C =
    1    0    0    0    0    0
    0    1    0    0    0    0
    0    0    1    0    0    0
    0    0    0    1    0    0
    0    0    0    0    1    0
    0    0    0    0    0    1

D =
    0    0
    0    0
    0    0
    0    0
    0    0
    0    0
```

Figure 2: The computed A, B, C and D matrices in MATLAB

```

syms q1 q2 dq1 dq2 a1 a2 real % states
syms u1 u2 real % inputs

x = [q1; q2; dq1; dq2; a1; a2];
u = [u1; u2];

% --- parameters chosen so that A has integer entries ---
m1 = 1; % mass/inertia joint 1
b1 = 2; % damping joint 1
k1 = 10; % stiffness joint 1

m2 = 1; % mass/inertia joint 2
b2 = 1; % damping joint 2
k2 = 3; % stiffness joint 2

T = 0.1; % actuator time constant (both actuators)

% Joint 1: m1*ddq1 + b1*dq1 + k1*q1 = tau1, tau1 = a1
ddq1 = (-b1*dq1 - k1*q1 + a1) / m1;

% Joint 2: m2*ddq2 + b2*dq2 + k2*q2 = tau2, tau2 = a2
ddq2 = (-b2*dq2 - k2*q2 + a2) / m2;

% Actuator dynamics: dot(a_i) = -1/T * a_i + 1/T * u_i
da1 = -1/T * a1 + 1/T * u1;
da2 = -1/T * a2 + 1/T * u2;

```

Figure 3: Defining State and Input Variables in MATLAB

```

fprintf('\nControllability analysis\n');
Co = ctrb(A,B);
rankCo = rank(Co);
n = size(A,1);
fprintf('Rank of the controllability matrix = %d (n = %d)\n', rankCo, n);

if rankCo == n
    disp('The controllability matrix has full rank.');
```

```

    disp('Thus the system is completely controllable.');
```

```

else
    disp('The controllability matrix does not have full rank.');
```

```

    disp('Thus the system is not completely controllable.');
```

```

end

fprintf('\nObservability analysis\n');
Ob = obsv(A,C);
rankOb = rank(Ob);
fprintf('Rank of the observability matrix = %d (n = %d)\n', rankOb, n);

if rankOb == n
    disp('The observability matrix has full rank.');
```

```

    disp('Thus the system is completely observable.');
```

```

else
    disp('The observability matrix does not have full rank.');
```

```

    disp('Thus the system is not completely observable.');
```

```

end

```

Figure 4: Defining and Computing the Controllability and Observability Matrices



All eigenvalues have strictly negative real parts.  
Thus the system is internally asymptotically stable.  
Thus the system is LTI and proper, it is also BIBO stable.

Controllability analysis  
Rank of the controllability matrix = 6 ( $n = 6$ )  
The controllability matrix has full rank.  
Thus the system is completely controllable.

Observability analysis  
Rank of the observability matrix = 6 ( $n = 6$ )  
The observability matrix has full rank.  
Thus the system is completely observable.

Figure 5: Controllability, Observability and Stability Analysis

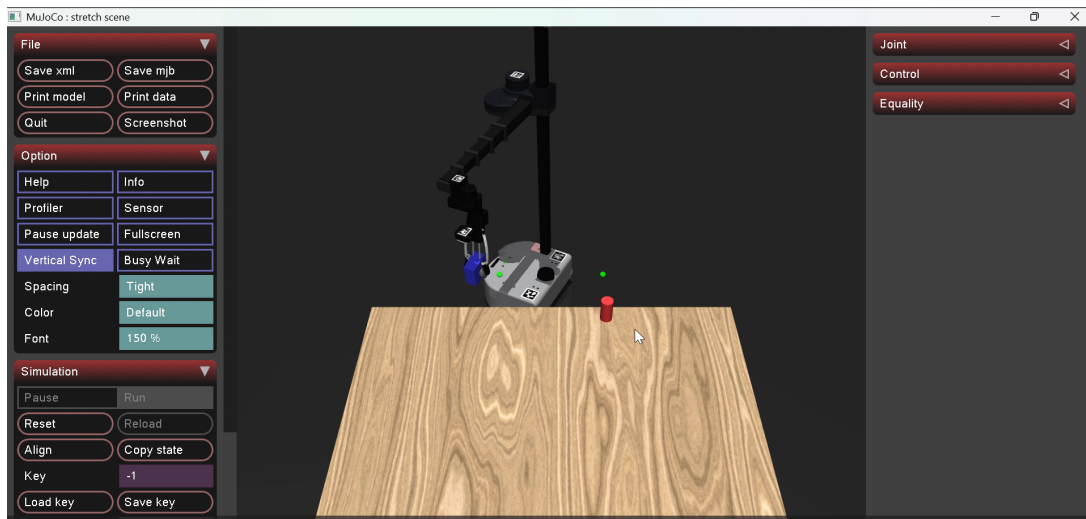


Figure 6: The Stretch3 robot picking a cuboidal object in MuJoCo

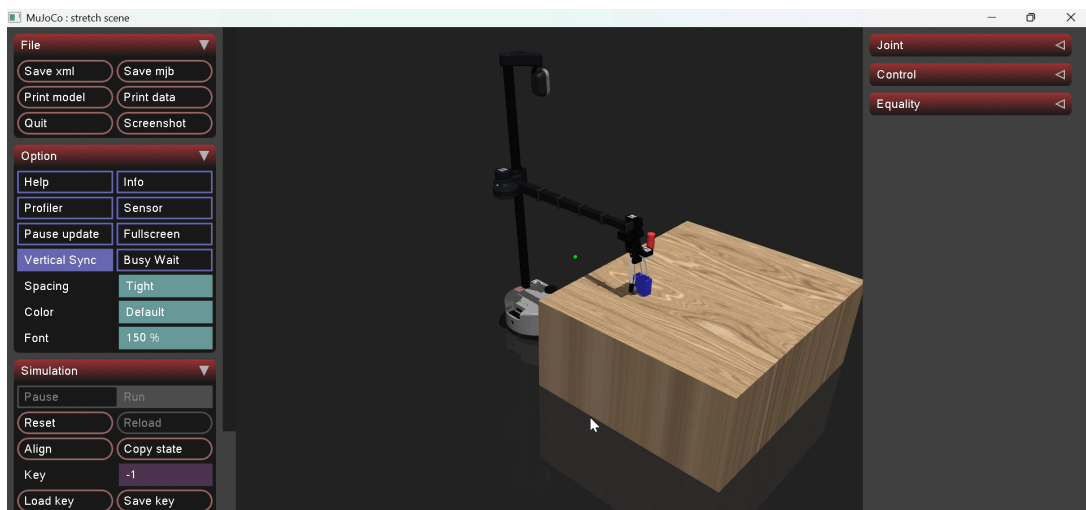


Figure 7: The Stretch3 robot places a cuboidal object in MuJoCo

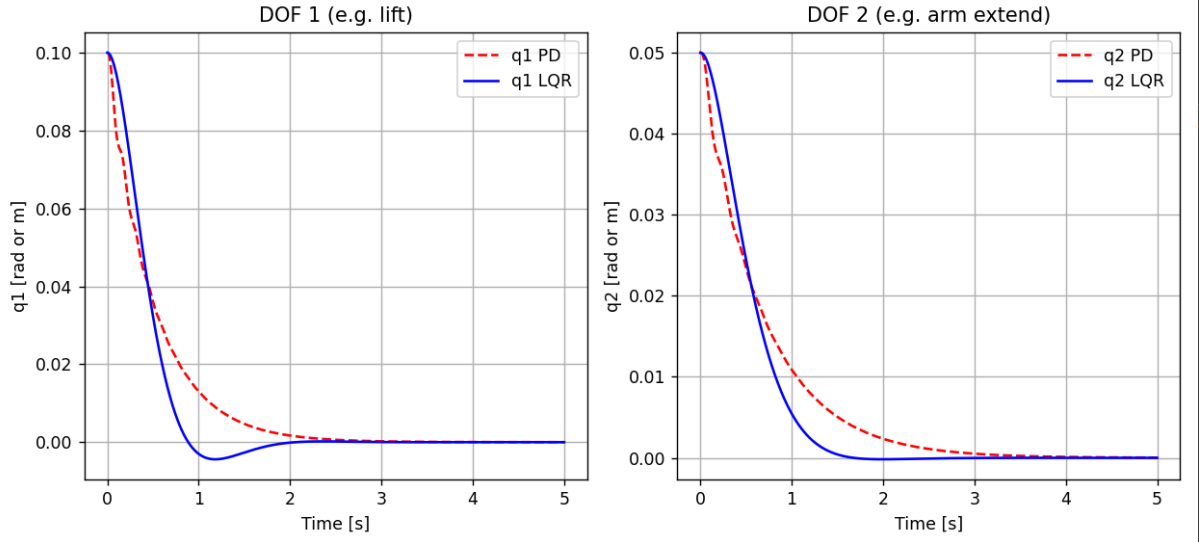


Figure 8: PD Controller vs LQR controller showing the joint positions vs time

## 8 References

- [1] S. Ljasenko, N. Lohse, and L. Justham, *A comparison of the manufacturing resilience between fixed automation systems and mobile robots in large structure assembly*, *Procedia CIRP*, vol. 57, pp. 235–240, 2016. doi:10.1016/j.procir.2016.11.041
- [2] D. Lukač, *Simulation of a pick-and-place cube robot by means of the simulation software KUKA Sim Pro*, in *2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, pp. 846–849, IEEE, 2018. doi:10.23919/MIPRO.2018.8400156
- [3] Hello Robot, *Stretch MuJoCo Simulation Repository*, [https://github.com/hello-robot/stretch\\_mujoco](https://github.com/hello-robot/stretch_mujoco)
- [4] Hello Robot, *Stretch AI Repository*, [https://github.com/hello-robot/stretch\\_ai](https://github.com/hello-robot/stretch_ai)
- [5] DeepMind, *MuJoCo Menagerie: Hello Robot Stretch 3*, [https://github.com/google-deepmind/mujoco\\_menagerie/tree/main/hello\\_robot\\_stretch\\_3](https://github.com/google-deepmind/mujoco_menagerie/tree/main/hello_robot_stretch_3)